Observability — Advanced Companion

Staff / Principal-level deep dives · For 14+ YOE engineers preparing for senior interviews

This companion goes beyond fundamentals. It covers the architectural depth, edge cases, and modern frontiers that staff and principal-level interviewers probe. Pair this with the main *Observability Interview Prep* document.

Reading order

- **Foundational track** (1–6): Service mesh, HA stack, TSDB internals, SLO calculus, multi-tenancy.
- **Modern frontier** (7–11): LLM/GenAI observability, eBPF networking, continuous profiling, AIOps, OTel deep internals.
- **Adjacent & orgs** (12–18): RUM, databases, streaming, edge/mobile, chaos, FinOps, ODD, postmortem culture.
- **Practice** (19–20): Advanced question bank + scenario-based design drills.

Contents

1. Service Mesh Observability

Why It Comes Up in Senior Interviews

Service meshes give you **L7 telemetry for free** — no code changes — but introduce sidecar overhead, complexity, and observability of the mesh itself. Interviewers want to see you weigh these trade-offs against SDK-based instrumentation.

What a Mesh Gives You Out of the Box

- **Per-service RED metrics** at the proxy layer (Envoy stats: upstream_rq_*).
- **Distributed tracing** — proxy adds B3/W3C headers, emits spans to OTel/Zipkin.
- **L7 access logs** in JSON.
- **mTLS visibility** — auth events, certificate rotation, identity-aware logs.
- **Topology** — Kiali/Hubble auto-discover service graphs from real traffic.

Sidecar vs Ambient Mesh

Aspect	Sidecar (Istio classic, Linkerd)	Ambient / Sidecarless (Istio ambient, Cilium
Memory	~50–150 MB per pod sidecar	Per-node ztunnel; 5–10x lower per workload
L7 visibility	Always available	Optional waypoint proxy
Upgrade	Pod restart per upgrade	Node-level upgrade
Trade-off	Heavy but mature	Lighter, newer, fewer features

Mesh + OTel — The Layered Picture

Best-in-class teams use both. The mesh provides **service-level** RED metrics (cheap, automatic). OTel SDKs add **business-level** spans (cart_id, tenant_id, decision branches). The mesh trace becomes the parent span; SDK adds children with semantic richness.

Observing the Mesh Itself

- Control plane (istiod) health: pilot xDS push latency, config errors.
- Sidecar resource use: envoy.server.memory_allocated, CPU.
- Certificate expiry alerts.
- Mesh policy denial counts (Allow vs Deny in AuthZ).

Read: Istio observability docs (istio.io/latest/docs/tasks/observability/), Cilium Hubble docs (docs.cilium.io/en/stable/observability/hubble/), Linkerd metrics reference.

Self-Check

- When would a mesh be more expensive than just running an OTel agent DaemonSet?
- Mesh trace propagation requires what specific app behavior to actually work?
- How do you detect a noisy neighbor in a multi-tenant mesh?

2. High-Availability Observability Stack

Why Prometheus Alone Fails Above ~10M Active Series

- Single-node TSDB — no replication, no horizontal scale.
- Retention bounded by disk size (typically 15–30 days).
- Federation creates query fan-out problems and label collisions.
- WAL replay on restart can take 30+ minutes for large heads.

The Three Mainstream Solutions — Cheat Sheet

	Thanos	Mimir	VictoriaMetrics
Model	Sidecar + object store	Microservices + S3	Cluster on block storage
Ingest	Via Prometheus + sidecar	Direct remote_write	Direct remote_write
Storage	S3 (cheapest)	S3	Block storage (HDD/SSD)
Multi-tenancy	Basic	Strong (enterprise grade)	Built-in
Query language	PromQL	PromQL	MetricsQL (superset)
Ops complexity	Medium	High (many components)	Low (1 or few binaries)
Best for	Existing Prom users, LTS	Large platform teams, 500+ devs	Most teams, cost-conscious
Compression	2–4x	Similar to Prom	Up to 10x

Thanos Components (Be Able To Sketch These)

- **Sidecar** — uploads Prom blocks to object store, serves recent data.
- **Store Gateway** — serves historical blocks from S3.
- **Querier** — fans out to sidecars + store gateways, dedupes HA pairs.
- **Compactor** — compacts, downsamples (5m, 1h), enforces retention.
- **Ruler** — evaluates recording & alerting rules against the global view.
- **Receive** (optional) — push-based ingestion, replaces sidecar model.

Mimir's Microservices (the parts you must name)

- **Distributor** — routes incoming series, validates, replicates.
- **Ingester** — in-memory write path, flushes blocks to object store.
- **Store-gateway** — historical reads from object store.
- **Querier / Query-frontend** — splitting, caching, sharding.
- **Compactor / Ruler / Alertmanager** — same roles as Thanos.

HA Pairs & Deduplication

Run two Prometheus replicas with identical scrape configs. Tag with replica=A and replica=B. The querier deduplicates on read. This protects against single Prom loss without compromising correctness.

Alertmanager Clustering

Alertmanager uses a **gossip protocol** (memberlist) to deduplicate and silence across replicas. Production runs 3 replicas minimum. Gossip port (9094) must be open. Without clustering, you get duplicate pages.

Senior-level trap: *"How does Alertmanager guarantee an alert fires exactly once across a cluster?"* Answer: it doesn't *guarantee* — gossip is eventually consistent. At-least-once is the contract; deduplication uses a fingerprint over labels.

Self-Check

- Draw Thanos full architecture on a whiteboard from memory.
- Why does Mimir scale better for multi-tenant SaaS than Thanos?
- When would VictoriaMetrics beat both? When would it lose?
- Walk through a Thanos compactor failure — what breaks, in what order?

3. Time-Series Database Internals

Why This Separates Senior From Staff

Knowing TSDB internals lets you diagnose *why* Prometheus is OOMing, why a query is slow, why retention is bloating. Most engineers can't. This question often shows up as: *"Walk me through what happens when Prometheus scrapes a target."*

Prometheus TSDB Block Layout

- **Head block** — in-memory, holds current 2-hour window.
- **WAL** (Write-Ahead Log) — durability for the head; replayed on restart.
- **Persistent blocks** — 2h immutable blocks on disk, eventually compacted.
- **Block contents** — chunks/ (compressed samples), index (postings, label inverted index), meta.json, tombstones.
- **Compaction** — 2h → 12h → 24h → 14d, deduplicates and reduces overhead.

Chunk Encoding

Samples encoded with **XOR encoding for floats** (Gorilla compression) and **varint encoding for timestamps**. A scraped sample averages ~1.3 bytes on disk. Gauges compress less efficiently than counters because of value variation.

The mmap Reality

Prometheus mmaps block index files. The kernel page cache decides what's in RAM. **Implication:** RSS reported by ps is misleading; actual working set depends on query patterns. Long-range queries pull cold blocks into cache and can push out the hot head.

Why High-Cardinality Queries Are Expensive

Postings list intersections grow as *O(min cardinality)*. A query with {job="x", instance=~".+"} over a high-cardinality instance label forces millions of postings list reads. Index is loaded; head series count balloons.

VictoriaMetrics Differences

- Custom **columnar storage** rather than chunks-per-series.
- Better compression for label-heavy data.
- MetricsQL adds rollup/aggregation functions PromQL lacks.
- Trade-off: no object storage; needs durable block storage.

Self-Check

- What's the WAL? How does it bound RPO on Prom crash?
- Why does histogram_quantile over a large time range get slow?
- Explain how Gorilla XOR encoding compresses floats.
- Walk through what happens from scrape → query for a single sample.

4. SLO Calculus — The Hard Parts

The Compounding Problem

If your service depends on three internal services each at 99.9%, your *maximum theoretical availability* is 99.9%^3 = 99.7%. **You cannot promise more than your weakest link.** This is why senior SLO conversations always start with dependency mapping.

Math: Dependent Service SLOs

Sequential dependency (must call A then B):
Availability = A_av × B_av
Redundant dependency (either A or B works):
Availability = 1 – (1 – A_av) × (1 – B_av)

```
Example with three sequential 99.9% deps: 0.999 × 0.999 × 0.999 = 0.997002999 -> 99.7%
Monthly budget shrinks from 43.2 min to 130 min Redundant pair, each 99.5%: 1 - (0.005 ×
0.005) = 0.999975 -> 99.997% Budget grows substantially with redundancy.
```

Window Type — Rolling vs Calendar

Window	Pros	Cons
Rolling 28 days	Smooth, no "month start" reset gaming	Slightly harder to communicate
Calendar month	Easy to report, customer-friendly	Bad incident on day 30 has no consequence
Multi-window burn	Captures both fast and slow burns	More PromQL complexity

Request-Based vs Window-Based SLOs

- **Request-based:** good requests / total requests. Naturally weights by traffic.
- **Window-based:** percentage of *minutes* the service was healthy. Low-traffic windows distort.
- **Senior insight:** For user-facing services, request-based is better; window-based is for batch and async work.

Composite SLOs

A "checkout works" SLO is composed of: cart-service availability + price-service freshness + payment-gateway availability + email-confirmation latency. Each contributes; weight them by user impact. Tools like **Sloth** and **Pyrra** (Kubernetes-native) generate the recording rules and burn-rate alerts from a YAML SLO spec.

Realistic SLO Targets

SLO	Allowed downtime / month	Allowed downtime / year
99%	7h 18m	3d 15h
99.5%	3h 39m	1d 19h
99.9% (three 9s)	43m 49s	8h 45m
99.95%	21m 54s	4h 22m
99.99% (four 9s)	4m 22s	52m 35s

SLO	Allowed downtime / month	Allowed downtime / year
99.999% (five 9s)	26 seconds	5m 15s

Bar talk: "Five 9s" is essentially impossible for a service with humans in the loop. Pushing toward 100% costs exponentially more for diminishing user benefit. Negotiate down, not up.

Self-Check

- Your service depends on five 99.9% services. What's the max SLO you can promise?
- Why is a 99.99% SLO usually a bad idea for a new service?
- Define an SLO for a batch job that runs every 6 hours.
- How does Sloth or Pyrra simplify SLO management?

5. Multi-Tenancy at Platform Scale

The Problem

Your observability platform serves 50 internal teams. Without isolation, one team's bad metric (say, label user_email) kills the platform for everyone. Senior interviews probe your strategy for fair-share, isolation, and chargeback.

Isolation Layers

Layer	Mechanism
Network	Per-tenant ingress, mTLS auth
Identity	<code>X-Scope-OrgID</code> header (Mimir/Loki/Tempo standard)
Query	Per-tenant query limits, time-range caps
Storage	Per-tenant directories in object store, per-tenant retention
Resource	Per-tenant cardinality and ingest rate limits
Cost	Per-tenant usage accounting → chargeback

Mimir / Loki / Tempo Tenant Headers

All three Grafana stack components honor X-Scope-OrgID on read and write. An OTel Collector gateway can inject this header per-source using the attributes or transform processor before forwarding to the storage backend.

Per-Tenant Limits to Enforce

- **Ingestion rate** — samples/sec, bytes/sec.
- **Cardinality** — max active series per tenant.
- **Query** — max parallel queries, max series fetched, max samples scanned.
- **Time range** — max query length.
- **Retention** — per-tenant retention policy.

Fair-Share Algorithms

Mimir uses **shuffle sharding** — each tenant maps to a subset of ingesters/queriers such that noisy neighbors can only affect a bounded fraction of co-tenants. Better than uniform sharding for blast radius control.

Chargeback / Showback

- Track per-tenant series count, ingest volume, query CPU as metrics.
- Convert to dollar cost using infrastructure spend model.
- **Showback first** (visibility), **chargeback later** (real billing) — sociotechnical lesson.
- Surface in Grafana dashboards per team; trigger conversations.

Self-Check

- Walk through how a request flows from tenant A through Mimir with shuffle sharding.

- How do you stop a tenant from accidentally DoS'ing the cluster with a regex query?
- Design a chargeback metric that's fair across spiky and steady workloads.

6. OpenTelemetry Deep Internals

Resource Detection

Every span/metric/log carries a **Resource** — attributes describing the source (service.name, k8s.pod.name, cloud.region). OTel SDKs auto-detect from env vars, K8s downward API, cloud metadata APIs (AWS IMDS, GCP metadata server). Custom ResourceDetector implementations let you add internal concepts (tenant, environment tier).

OTLP Wire Format

Aspect	OTLP/gRPC	OTLP/HTTP
Encoding	Protobuf	Protobuf or JSON
Performance	Faster, multiplexed	Slightly slower, simpler
Port	4317 (default)	4318 (default)
Use when	Backend-to-backend	Browser, edge, firewalls

Context Propagators

OTel ships pluggable propagators. Multiple can be enabled simultaneously:

- **tracecontext** — W3C standard (traceparent, tracestate).
- **baggage** — W3C baggage for arbitrary k/v travel.
- **b3** / **b3multi** — Zipkin compatibility.
- **jaeger** — for legacy Jaeger systems.
- **xray** — AWS X-Ray.

Exemplars — Linking Metrics → Traces

An **exemplar** is a single trace ID attached to a metric data point. Stored in Prometheus from OTLP histograms. In Grafana you click a p99 latency spike and jump to a sample slow trace. Requires both Prometheus and exporter exemplar support enabled.

```
# Prometheus --enable-feature=exemplar-storage # OTel Collector exporter exporters: prometheusremotewrite: endpoint: http://prom:9090/api/v1/write send_metadata: true enable_exemplars: true
```

Schema URL & Semantic Versioning

SchemaURL on a Resource declares which version of OTel semantic conventions applies. Lets backends auto-translate between schema versions during a migration. Critical for long-lived platforms — semconv evolves and breaking changes happen.

Processors Worth Memorizing

Processor	Use
batch	Batch before export — required for efficiency
memory_limiter	Drop or back-pressure when RAM threshold hit (first in pipeline)
attributes	Add / update / hash / delete attributes

Processor	Use
resource	Modify Resource attributes (e.g. add cluster name)
transform (OTTL)	Powerful per-record transformation language
filter	Drop records matching expression
tail_sampling	Decide span retention after seeing whole trace
probabilistic_sampler	Head sampling by hash of trace ID
k8sattributes	Auto-enrich with K8s metadata (pod, namespace, labels)
routing	Route to different exporters by tenant or attribute

Self-Check

- How do exemplars get from app → histogram → Prometheus → Grafana?
- Why might you enable both tracecontext and b3 propagators?
- What's the k8sattributes processor doing under the hood?
- When would you switch from OTLP/gRPC to OTLP/HTTP?

7. LLM & GenAI Observability

Why It's a Hot Topic in 2026 Interviews

LLMs are first-class production infrastructure. Senior interviewers — especially at AI-adjacent companies — will ask how you'd observe latency, cost, quality, and safety of GenAI systems. OTel's gen_ai.* semantic conventions standardized in late 2025 are the modern answer.

Why LLM Observability Differs From APM

- **Non-determinism** — same input → different outputs. Can't reproduce bugs without exact prompt, params, seed.
- **Token-based cost** — latency and bill correlate with tokens, not requests.
- **Prompt sensitivity** — PII risk if you log raw prompts.
- **Quality dimension** — "correct" is fuzzy; you need eval signals (hallucination, relevance).
- **Multi-step** — chains, tools, retrieval; each step needs its own span.

OTel GenAI Semantic Conventions (gen_ai.*)

Attribute	Meaning
gen_ai.system	Provider (openai, anthropic, bedrock)
gen_ai.request.model	Requested model name
gen_ai.response.model	Actual model that responded
gen_ai.request.temperature	Sampling temperature
gen_ai.usage.input_tokens	Prompt token count
gen_ai.usage.output_tokens	Completion token count
gen_ai.response.finish_reasons	stop / length / tool_calls / content_filter
gen_ai.operation.name	chat, completion, embeddings

Where to Put the Prompt/Response Content

Anti-pattern: raw prompts in span attributes — they're indexed, size-bounded, and expose PII. **Pattern:** use **span events** with gen_ai.content.prompt and gen_ai.content.completion. Events can be dropped or redacted at the Collector without code changes.

Key LLM Metrics

- **Latency:** time-to-first-token (TTFT), time-per-output-token (TPOT), total e2e.
- **Cost:** tokens × price; per-tenant, per-feature.
- **Quality:** eval scores (relevance, groundedness, toxicity).
- **Reliability:** error rate, rate limit hits, retries.
- **Drift:** embedding distribution shifts, output length distribution shifts.

Tools & Libraries

- **OpenLLMetry** (Traceloop) — OTel-native auto-instrumentation for OpenAI, Anthropic, LangChain, etc.
- **Langfuse** — open-source LLM observability; OTel-compatible.
- **Arize Phoenix** — open-source LLM tracing & eval.
- **OpenObserve** — OTLP-native, supports gen_ai conventions.
- **Datadog LLM Obs / Honeycomb / New Relic** — vendor support landing in 2025–2026.
- **MLflow** — supports gen_ai semconv with MLFLOW_ENABLE_OTEL_GENAI_SEMCONV.

RAG Observability Specifics

A RAG trace should have these spans (parent → children):

- **query_received** (parent) — user query, request_id.
- **embedding_generation** — model, latency, tokens.
- **vector_search** — k, distance metric, retrieved doc IDs, scores.
- **context_assembly** — selected chunks, total context tokens.
- **llm_inference** — gen_ai.* attributes, finish_reason.
- **response_returned** — output tokens, total latency.

Read: opentelemetry.io/blog/2025/ai-agent-observability/ · openobserve.ai/blog/opentelemetry-for-llms/ · uptrace.dev/blog/opentelemetry-ai-systems

Self-Check

- Why store prompt content in span events, not attributes?
- Sketch a trace for a RAG question-answer flow.
- What's the difference between TTFT and TPOT? Why both matter?
- How do you detect prompt-injection attempts via observability?

8. AI Agent & MCP Observability

The Challenge

Agents are multi-step LLM chains with tool calls and memory. A single user request becomes a tree of decisions: "think → call tool → read result → think → call another → respond." Without observability, debugging "why did the agent loop forever?" is impossible.

Agent Trace Anatomy

- **agent_invocation** (root span) — user input, agent name, goal.
- **agent_planning** — reasoning step, thought, plan.
- **tool_call** — tool name, input args, output, latency, success.
- **memory_read / memory_write** — namespace, key, vector similarity.
- **handoff** — agent-to-agent delegation (multi-agent systems).
- **llm_call** — gen_ai.* on each model invocation.

OTel GenAI SIG Agent Conventions (Experimental)

Two parallel efforts being standardized in 2026:

- **AI Agent Application Conventions** — tasks, actions, memory, artifact tracking.
- **AI Agent Framework Conventions** — framework-specific instrumentation for CrewAI, AutoGen, LangGraph, Semantic Kernel.

MCP (Model Context Protocol) Observability

MCP servers expose tools to LLM clients (Claude, agents, IDEs). Observability for MCP focuses on:

- **Tool call latency** — per tool, per server.
- **Tool success rate** — schema validation failures, auth errors.
- **Tool usage frequency** — which tools matter; deprecate dead ones.
- **Per-client usage** — chargeback, rate limiting.
- **Permission audit logs** — every tool invocation logged with caller identity.

Agent Eval Signals

- **Task completion rate** — did the agent achieve the goal?
- **Loop detection** — same tool with same args called N+ times.
- **Hallucinated tool calls** — calling tools that don't exist.
- **Cost per task** — total tokens across the chain.
- **Steps to completion** — distribution per task type.

Senior insight: Agent observability isn't just APM with an LLM flavor. You're observing an *autonomous decision-making system*. The eval and quality dimensions are as important as latency and cost.

Self-Check

- Sketch the trace tree for an agent that calls 3 tools to answer a question.
- How would you detect an agent caught in a tool-call loop?
- What's the MCP-specific equivalent of an API rate limit dashboard?

9. eBPF Networking Observability

Why eBPF Changes the Game

eBPF runs sandboxed programs in the Linux kernel — at syscall, networking, and tracepoint hooks. For observability: **zero-code-change L7 visibility** without sidecars or app instrumentation.

Cilium + Hubble

- Hubble exposes flow logs: src/dst service, verdict (allow/deny), protocol, latency.
- HTTP/gRPC/Kafka L7 visibility from a single DaemonSet — no sidecars.
- Service map auto-built from real traffic.
- Network policy observability — see which policy allowed/denied each flow.

Common eBPF Observability Tools

Tool	Use
Cilium / Hubble	L3-L7 network observability + policy
Pixie	Auto-instrument K8s apps, including DB queries
Parca / Pyroscope	Always-on CPU profiling
Inspektor Gadget	Ad-hoc debugging primitives
bpftrace / bcc	Custom probes, scripting
Tetragon	Security observability — process exec, syscall trace

Trade-offs

- **Pros:** zero app changes, kernel-level truth, low overhead (1–3%).
- **Cons:** needs recent kernel (≥5.4 ideal), platform-specific, less semantic context than SDK spans.
- **Best for:** L3/L4 metrics, security, profiling, ad-hoc debugging.
- **Worse for:** business semantics — eBPF doesn't know what "cart_id" is.

Read: docs.cilium.io/en/stable/observability/ · isovalent.com/blog (Cilium creators) · Brendan Gregg's BPF Performance Tools book.

Self-Check

- Where would eBPF win vs OTel SDK? Where would it lose?
- Walk through what happens when Hubble captures a Pod-to-Pod HTTP call.
- Why is kernel version critical for eBPF observability?

10. Continuous Profiling — The Fourth Pillar

What Profiling Adds

Logs/metrics/traces tell you *that* something is slow. Profiles tell you *which function* consumes CPU/memory/locks. Always-on profiling at low sample rate (~100 Hz) adds <1% overhead and answers questions you couldn't even ask before.

Profile Types

- **CPU** — sampled stack traces; what's hot.
- **Allocations** — what's allocating; GC pressure source.
- **Heap (live)** — what's retained; leak detection.
- **Lock contention** — what's blocking threads.
- **I/O wait, off-CPU** — why threads aren't running.

pprof — The De Facto Format

Google's pprof format is the open standard. Most languages can emit it: Go natively, Java via async-profiler, Python via py-spy, Ruby via rbspy, .NET via dotnet-trace. Tools that consume it: **Parca**, **Pyroscope**, **Grafana Profiles**, **Google Cloud Profiler**, **Polar Signals**.

Flame Graphs

Visualization invented by Brendan Gregg. X-axis = sample count (width = time spent). Y-axis = stack depth. **Read left-to-right; look for wide plateaus at the top — those are the hot leaf functions.**

Differential / Comparison Profiles

Compare today's flame graph vs yesterday's, or post-deploy vs pre-deploy. Highlights regressions down to specific functions. Game-changer for performance review of new releases.

Profile-Trace Linking

Add trace_id and span_id as pprof labels. Now Grafana lets you click from a slow trace → flame graph for that exact request. Implemented in async-profiler, Pyroscope, Grafana Pyroscope SDK.

Self-Check

- Read a flame graph: what's the difference between a wide top and a wide middle?
- Why is allocation profiling sometimes more useful than CPU profiling?
- How does pprof label injection enable trace→profile linking?

11. AIOps, Anomaly Detection & LLM-Assisted RCA

Where ML Genuinely Helps Today

Use case	Technique	Maturity
Metric anomaly detection	Holt-Winters, Prophet, isolation forest	Production-ready
Seasonal forecasting	STL decomposition	Production-ready
Log clustering / template mining	Drain3, LogPAI	Production-ready
Alert correlation / grouping	Topology + temporal proximity	Mature in BigPanda, Moogsoft
Incident summarization	LLM over logs/timeline	Improving fast (2025–2026)
Suggested queries / dashboards	LLM + telemetry context	Improving
Auto-remediation	Rule + ML hybrid	Early — narrow domains only

Where ML Falls Short

- **True root-cause inference** remains weak — correlation isn't causation.
- **Incident prediction** beyond hours is mostly noise.
- **Black-box anomaly detection** can flag a thousand low-value anomalies.
- **Replacing on-call** — no system is good enough to wake itself up reliably yet.

Modern LLM-Assisted Patterns

- **Postmortem draft generation** — feed timeline, logs, chat → LLM drafts a postmortem.
- **BubbleUp-style anomaly exploration** (Honeycomb) — high-dimensional pivots.
- **Natural language query** — "show me 5xx errors in checkout for users in EU yesterday."
- **Diff-aware deploy analysis** — LLM cross-references deploy diff with metric anomaly.

Anomaly Detection Math (be prepared to explain)

Z-score: $z = (x - \mu) / \sigma$. Threshold $z > 3$.
EWMA: exponentially weighted moving average — newer points weighted higher.
Holt-Winters: three components — level, trend, seasonality. Built into Grafana, Prometheus has holt_winters().
Isolation Forest: tree-based; outliers isolated in fewer splits.

Self-Check

- Why isn't z-score enough for production metric anomaly detection?
- Explain what Drain3 does and why it helps log analysis.
- What are the failure modes of LLM-generated postmortems?

12. Database & Stateful Workload Observability

Why Stateful Is Harder

Stateless services follow RED. Stateful services need **per-query, per-shard, per-replica** visibility — and recovery scenarios that don't exist elsewhere (replica catch-up, snapshot restore, split-brain detection).

Postgres — What to Watch

- **pg_stat_statements** — query plan, exec count, mean time.
- **pg_stat_activity** — running queries, lock waits, idle-in-transaction.
- **Replication lag** — pg_stat_replication.lag.
- **Connection pool saturation** — pgbouncer metrics.
- **Cache hit ratio** — >99% target; below 95% is alarm.
- **Bloat** — table and index bloat; vacuum lag.
- **Lock waits** — pg_locks; deadlock count.

MySQL Equivalents

- performance_schema events_statements_summary_by_digest.
- InnoDB buffer pool hit ratio.
- Replication lag (Seconds_Behind_Master).
- Long-running queries via slow query log.

Redis Observability

- **SLOWLOG** — queries exceeding threshold.
- **Memory fragmentation ratio** — >1.5 is concerning.
- **Evictions / sec** — non-zero on non-LRU keys is a problem.
- **Replication lag** — master_repl_offset diff.
- **Persistence** — RDB/AOF write latency, last save success.

Kafka Observability

- **Consumer lag** per consumer group per partition — most-asked Kafka metric.
- **Under-replicated partitions** — broker health canary.
- **ISR shrinks** — network or disk issue indicator.
- **Request latency histograms** per request type.
- **Disk usage** per broker — log retention math.
- **End-to-end latency tracing** — producer span → broker → consumer span (via OTel Kafka instrumentation).

Exporters to Know

DB	Prometheus exporter
Postgres	postgres_exporter (prometheus-community)

DB	Prometheus exporter
MySQL	mysqld_exporter
Redis	redis_exporter (oliver006)
Kafka	jmx_exporter or strimzi-kafka-exporter
MongoDB	mongodb_exporter
Elastic/Opensearch	elasticsearch_exporter

Self-Check

- Diagnose: queries are slow, CPU is fine, what do you look at?
- How do you observe consumer lag in a way that's actionable for alerting?
- Postgres replication lag — what's normal, what's not, why?

13. Frontend / RUM & End-to-End Tracing

Why It Matters

Backend can be 100ms p99 and user experience can still be terrible. RUM (Real User Monitoring) captures the actual experience in the browser/app. Senior interviews increasingly probe full-stack observability ownership.

Core Web Vitals (Google standards)

Metric	What it measures	Good
LCP (Largest Contentful Paint)	Time to render largest element	< 2.5s
INP (Interaction to Next Paint)	Replaces FID; full interaction responsiveness	< 200ms
CLS (Cumulative Layout Shift)	Visual stability	< 0.1
TTFB (Time to First Byte)	Server response time	< 800ms
FCP (First Contentful Paint)	First content visible	< 1.8s

Browser Tracing

OTel Browser SDK emits spans for fetch, XMLHttpRequest, document load, user interactions. Propagates traceparent header to the backend → unified trace.

```
const provider = new WebTracerProvider(); provider.addSpanProcessor(new
BatchSpanProcessor(new OTLPTraceExporter({ url:
'https://otel-gateway.example.com/v1/traces', }))); registerInstrumentations({
instrumentations: [ new FetchInstrumentation({ propagateTraceHeaderCorsUrls:
[/.*\.example\.com/], }), ], });
```

Session Replay & Error Tracking

- **Sentry, Datadog RUM, LogRocket, FullStory** — capture errors with breadcrumbs.
- **Session replay** — DOM-level reconstruction, valuable but heavy on bandwidth.
- **Source maps** are essential for readable stack traces in production.

End-to-End Trace Stitching

Browser span (Span A) → API gateway span (Span B, child of A) → backend microservices (Spans C, D, E, children of B) → database (Span F, child of C). All sharing the same trace_id. Setting this up correctly requires CORS allowing traceparent header.

Self-Check

- What did INP replace in 2024? Why?
- Walk through propagating trace context from a React app to backend.
- Trade-offs of session replay in regulated industries?

14. Streaming & Event-Driven Observability

Why Async Breaks Tracing Defaults

OTel HTTP propagation is transparent. **Kafka, RabbitMQ, SQS, EventBridge** require explicit context injection into message headers/properties and extraction on consume. Without this, every async hop creates a new disjoint trace.

Kafka Tracing Pattern

```
// Producer ProducerRecord rec = new ProducerRecord<>(topic, key, value);
GlobalOpenTelemetry.getPropagators().getTextMapPropagator() .inject(Context.current(),
rec.headers(), KAFKA_HEADER_SETTER); producer.send(rec); // Consumer Context parent =
GlobalOpenTelemetry.getPropagators().getTextMapPropagator() .extract(Context.current(),
record.headers(), KAFKA_HEADER_GETTER); try (Scope s = parent.makeCurrent()) {
process(record); // spans here become children of producer span }
```

Consumer Lag — The Core SLI

Define **SLI = consumer_offset / producer_offset** per partition. SLO: "99% of partitions lag < 1000 messages over 5 minutes." Alert on lag growth rate, not absolute — a steady backlog during peak is normal; an accelerating backlog is not.

Flink Observability

- **Backpressure** — most important Flink signal; per-operator.
- **Checkpoint duration** & failure count.
- **End-to-end latency** — event time vs processing time gap.
- **Restart count** — job stability.
- **State size** — RocksDB/heap growth.

Exactly-Once Verification

Claims of exactly-once need observable proof. Track:

- Duplicate detection counts (idempotency keys).
- DLQ size — failed messages that need recovery.
- Checkpoint completion + transaction commit success ratio.
- End-to-end reconciliation via business invariants (e.g. money in = money out).

Self-Check

- Why is consumer lag alone misleading? What else do you alert on?
- How would you trace an event through Kafka → Flink → Postgres sink?
- What's a DLQ pattern and how do you observe its health?

15. Mobile / Edge Observability

Constraints

- **Bandwidth** — cellular/edge networks are expensive and unreliable.
- **Battery** — observability shouldn't drain devices.
- **Intermittent connectivity** — buffer locally, send opportunistically.
- **Privacy** — device IDs, location, PII all elevated risk.
- **Cold starts** — app launch time is critical UX metric.

Patterns

- **Buffer + batch** — collect locally, flush on WiFi.
- **Prioritization** — crashes always sent; debug logs dropped if bandwidth tight.
- **Summarization at edge** — pre-aggregate metrics before send.
- **Sampling** — aggressive on traces; full on errors.
- **Compression** — OTLP gzip on slow networks.

OTel Mobile SDKs

- **Android** — OTel Android SDK (alpha-ish), Embrace, Datadog.
- **iOS** — OTel Swift SDK, Embrace, Datadog, New Relic.
- **React Native / Flutter** — community OTel wrappers.

Edge / IoT

Lightweight Collector profile (OTel Collector with minimal receivers/exporters) running at edge. Forwards to regional gateways. Handles disconnections via file_storage extension for persistent queues.

16. Synthetic Monitoring & Chaos Engineering

Synthetic Monitoring

Active probing — you run scripted checks against your service continuously, from multiple regions. **Catches issues before real users do**, especially during low-traffic windows.

- **Blackbox Exporter** — Prometheus-native HTTP/TCP/DNS/ICMP probes.
- **Grafana Synthetic Monitoring / k6 Cloud** — managed, multi-region.
- **Checkly** — API + browser checks.
- **Datadog Synthetic** — vendor managed.

Synthetic SLI Pattern

Run a synthetic transaction every 30s from 5 regions. Define SLI as *fraction of synthetic probes that succeed within latency budget*. Provides traffic floor for SLO math during quiet hours.

Chaos Engineering — Hypothesis-Driven

Don't break things to break things. Frame each experiment as a hypothesis: *"If we kill a random pod, p99 latency stays under 500ms because the load balancer reroutes in <3s."* Run experiment, observe via your observability stack, validate or invalidate.

Chaos Tools

Tool	Strength
Chaos Mesh	K8s-native, visual UI, CNCF incubating
Litmus Chaos	K8s-native, hub of experiments
Gremlin	Commercial, full enterprise platform
Chaos Monkey	Original — Netflix EC2 random termination
Pumba	Docker container chaos

Game Days

Coordinated chaos exercises — schedule a window, run a scenario, validate runbooks and team response. Best-in-class teams run quarterly. Outcome: every drill exposes 2–3 gaps in alerts, runbooks, or training.

17. FinOps for Observability

The Conversation Everyone Has Now

Observability bills routinely cross seven figures. In 2025–2026, every senior interviewer asks about cost. You need a framework — not just "we drop debug logs."

Cost Attribution Hierarchy

- **Per service** — straightforward via service.name resource attribute.
- **Per team** — map services → teams via a service catalog tag.
- **Per signal** — metrics vs logs vs traces vs profiles breakdown.
- **Per environment** — prod vs staging vs dev (dev should be 1–5% of prod).
- **Per use case** — debugging logs vs audit logs vs business analytics.

Cardinality Budgets

Allocate cardinality (active series count) to each team like compute quota. When they hit it, they must justify expansion or drop labels. Surface usage in a Grafana dashboard. Creates self-policing without central gatekeeping.

Vendor vs Self-Host TCO

Factor	Vendor (Datadog, NR, Honeycomb)	Self-host (LGTM, ELK, VM)
Per-GB ingest cost	$0.10–$2.50 (high)	$0.005–$0.05 (storage + compute)
Ops headcount	Low — 0.1–0.3 FTE	Higher — 1–3 FTE for HA stack
Innovation speed	Faster — new features ship	Slower — open source pace
Lock-in	Risk on proprietary APIs	OTLP-native = portable
Best for	Small/mid team, time-to-value	Large scale, regulated, cost-sensitive

Cost-Reduction Playbook

- **Audit top-cardinality metrics** — usually 5% of metrics drive 80% of cost.
- **Drop noisy logs at source** — health checks, debug in prod, structured noise.
- **Convert chatty logs to metrics** — counters and histograms are 100x cheaper.
- **Tail-sample traces** — keep errors and slow paths; sample the rest.
- **Tier retention** — 7d hot / 30d warm / 1y cold.
- **Pre-aggregate** — recording rules to collapse high-cardinality reads.
- **Renegotiate / right-size vendor** — commit-and-discount with realistic forecast.
- **Evaluate self-host for log workloads** — Loki for K8s logs is usually 5–10x cheaper.

18. Observability-Driven Development & Postmortem Culture

Observability-Driven Development (ODD)

Charity Majors' framing: *instrument first, debug in production, design feature flags + observability together.* The core idea — you can't have a tight feedback loop on services you can't see.

- Every new feature ships with observability for its key behaviors.
- Engineers debug in prod against high-cardinality wide events, not local repros.
- Feature flags allow controlled rollouts; observability validates each step.
- Canary analysis is automated — metrics determine promote/rollback.

Canary Analysis

Compare a canary (5% traffic) against baseline (95%) on golden signals. Automated tools like **Argo Rollouts AnalysisRun**, **Flagger**, or **Spinnaker Kayenta** abort the rollout if canary metrics deteriorate beyond threshold.

Blameless Postmortems

- **Focus on systems, not individuals.** Replace "X made a mistake" with "the system allowed X to make a mistake."
- **Timeline** — what happened, when, who acted, what they knew at the time.
- **Contributing factors** — plural, not "root cause." Modern thinking rejects single root causes.
- **What worked well** — preserve the practices that saved you.
- **Action items** — concrete, owned, dated. Track to completion.
- **Distribute widely** — postmortems are organizational learning, not punishment.

Incident Severity Levels

SEV	Definition	Response
SEV-1	Critical user impact, outage	Page on-call, war room
SEV-2	Degraded but partially working	Page, no war room
SEV-3	Minor impact, internal only	Ticket, business hours
SEV-4	Cosmetic / no user impact	Backlog

Incident Command (ICS-lite)

- **Incident Commander (IC)** — coordinates response, makes calls.
- **Communications Lead** — owns external/internal updates.
- **Subject Matter Experts** — fix the issue.
- **Scribe** — writes timeline as it happens (often the same as IC).
- Rotate roles; don't let the senior eng default to IC every time.

Self-Check

- Walk through a postmortem you'd write. What headers, what sections?

- Why "contributing factors" instead of "root cause"?
- When does an SEV-2 escalate to SEV-1?

19. Advanced Question Bank (60+)

Service Mesh

- Compare sidecar vs ambient mesh from an observability standpoint.
- How do you trace through a service mesh without app instrumentation?
- What does Cilium Hubble see that an Envoy sidecar doesn't?
- Diagnose: app latency spike, but envoy sidecar shows nothing — where do you look?

HA Stack & TSDB

- Compare Thanos vs Mimir vs VictoriaMetrics on five dimensions.
- Why is Thanos sidecar problematic for some deployments?
- How does Mimir's shuffle sharding limit blast radius?
- Explain Prometheus WAL replay performance characteristics.
- How does Gorilla XOR encoding work?
- Walk through what happens during Prometheus compaction.

SLO Calculus

- Service depends on 4 services at 99.95%. Max SLO?
- When would window-based SLO be better than request-based?
- Why is composite SLO math non-trivial?
- How does Sloth generate alert rules from a YAML spec?
- Why 28-day rolling SLO vs calendar month?

Multi-tenancy

- How does X-Scope-OrgID isolate tenants in Mimir/Loki?
- What's shuffle sharding and why does it matter?
- Design chargeback for an internal observability platform.
- How would you stop a tenant from killing the cluster?

OTel Internals

- What are exemplars and how do they get from OTel to Grafana?
- Why is memory_limiter always the first processor?
- Explain SchemaURL and its role in semconv evolution.
- When use OTLP/HTTP vs gRPC?
- What does the k8sattributes processor add to a span?
- How is Resource different from span attributes?

LLM & GenAI

- Why store prompt content in span events not attributes?
- Sketch a RAG trace tree.
- Define an SLO for an LLM-powered customer support bot.
- How would you detect a prompt injection attempt?
- What's TTFT vs TPOT and why does the distinction matter?
- How would you observe token cost per feature?

Agents & MCP

- Sketch the trace for a 3-step agent with tool use.
- How do you detect an agent stuck in a loop?
- What's the MCP-specific equivalent of an API rate limit dashboard?
- Design eval signals for an autonomous agent.

eBPF & Profiling

- When does eBPF win over SDK instrumentation? When lose?
- Read a flame graph: wide top vs wide middle?
- What's pprof label injection for?
- Why does Cilium Hubble not need sidecars?
- How does always-on profiling stay under 1% overhead?

AIOps

- Why isn't z-score enough for production anomaly detection?
- Explain Drain3 log clustering.
- What are realistic LLM-assisted RCA use cases today?
- Failure modes of LLM-generated postmortems?

Database / Streaming

- Diagnose: queries slow, CPU fine — what next?
- How to alert on Kafka consumer lag actionably?
- Trace propagation through Kafka — code-level.
- What's a DLQ and how do you observe it?
- Why is exactly-once a claim that needs proof?

RUM / Frontend

- Explain INP. Why did it replace FID?
- Walk through trace propagation from React app to backend.
- Trade-offs of session replay in regulated industries?
- How do source maps make production errors actionable?

Mobile / Edge

- How do you handle observability during device offline periods?
- What signals get priority on a constrained mobile uplink?
- Why is OTel Collector with file_storage extension useful at edge?

Chaos / Synthetic

- Frame a chaos experiment as a hypothesis.
- Why is synthetic monitoring critical for low-traffic services?
- Walk through a game day — agenda, roles, outcomes.

FinOps

- Cut observability bill 40% — full playbook.
- Cardinality budget per team — how do you enforce it?

- Vendor vs self-host TCO comparison.
- When does Loki beat Elasticsearch on cost?

ODD & Culture

- What's observability-driven development?
- Why "contributing factors" instead of "root cause"?
- How do you make postmortems organizationally valuable?
- What separates SEV-1 from SEV-2?

20. Scenario-Based Design Drills

Drill 1 — Observability for a 200-microservice GenAI platform

- Multi-tenant — each customer is a tenant.
- Mix of latency-sensitive (chat) and batch (eval) workloads.
- Cost target: $50k/month observability budget total.
- Required signals: traces, metrics, logs, LLM-specific (token, cost, eval scores).
- Compliance: PII-aware logging (HIPAA, GDPR).
- *Cover:* Collector topology, gen_ai semconv adoption, tenant isolation in Mimir/Tempo, cost model.

Drill 2 — Cut observability cost from $2M/yr to $1M/yr

- Audit current state: 95% Datadog, 5% Prom for K8s.
- Identify top-cardinality offenders.
- Phase migration of K8s logs from Datadog to Loki.
- Convert noisy logs to metrics.
- Tail-sample traces, keep errors + p95+ latency only.
- Renegotiate Datadog with committed-use discount on remaining scope.
- *Cover:* migration risk, validation, rollback plan, team change management.

Drill 3 — Add observability to a legacy Java monolith going microservices

- 50 services on the migration roadmap.
- Existing: Splunk for logs, no traces, basic JMX metrics.
- *Cover:* OTel Java agent rollout phasing, log correlation via MDC, gradual Splunk → Loki migration, defining SLOs per new service.

Drill 4 — Build observability for a hybrid edge + cloud system

- 100k edge devices, 5 cloud regions.
- Edge devices: intermittent connectivity, 4G uplink.
- *Cover:* edge collector profile, batching/buffering, priority signals, edge → regional gateway → central backend, observability of the observability pipeline itself.

Drill 5 — On-call is drowning in 1000+ pages/week

- Inherited a noisy alerting system from a prior team.
- *Cover:* alert audit framework, SLO conversion plan, runbook coverage, kill criteria for noisy alerts, measuring success (pages/shift trend).

Drill 6 — Design SLOs for a multi-team checkout flow

- 5 services involved: cart, pricing, payment, inventory, email.
- Customer expects sub-2s checkout, 99.9% success.
- *Cover:* per-service SLI, composite SLO math, dependency mapping, error budget policy, escalation when budget depleted, cross-team negotiation.

Drill 7 — Add LLM observability to an existing OTel stack

- App is in Python, uses OpenAI + Anthropic + Bedrock.
- RAG over an internal vector store.

- *Cover:* OpenLLMetry adoption, gen_ai.* attributes, PII handling via Collector transforms, cost dashboards, eval signal integration.

21. Advanced Resources

Specifications & Standards

- **OpenTelemetry GenAI semantic conventions**: opentelemetry.io/docs/specs/semconv/gen-ai/
- **OpenTelemetry general semconv**: opentelemetry.io/docs/specs/semconv/
- **OTLP spec**: opentelemetry.io/docs/specs/otlp/
- **W3C Trace Context**: w3.org/TR/trace-context/
- **Prometheus exposition format**: prometheus.io/docs/instrumenting/exposition_formats/

Architecture Deep Dives

- **Thanos design**: thanos.io/tip/thanos/design.md/
- **Grafana Mimir architecture**: grafana.com/docs/mimir/latest/references/architecture/
- **VictoriaMetrics design**: docs.victoriametrics.com/single-server-victoriametrics/
- **Cilium docs**: docs.cilium.io/
- **Pyroscope/Grafana Profiles docs**: grafana.com/docs/pyroscope/latest/

LLM/GenAI Observability

- **OpenTelemetry AI Agent Observability blog**: opentelemetry.io/blog/2025/ai-agent-observability/
- **OpenLLMetry (Traceloop)**: github.com/traceloop/openllmetry
- **Langfuse**: langfuse.com — open source LLM obs.
- **Arize Phoenix**: phoenix.arize.com — open source LLM tracing & eval.
- **Datadog OTel GenAI integration blog**: datadoghq.com/blog/llm-otel-semantic-convention/
- **Uptrace AI systems guide**: uptrace.dev/blog/opentelemetry-ai-systems
- **OpenObservability Talks podcast** — S6E06 on OTel for GenAI.

Books — Advanced

- **Systems Performance, 2nd ed.** — Brendan Gregg (perf + USE method).
- **BPF Performance Tools** — Brendan Gregg (eBPF depth).
- **Database Internals** — Alex Petrov (TSDB context).
- **Designing Data-Intensive Applications** — Kleppmann (essential).
- **The Art of SLOs** course — Google (sre.google/workbook/implementing-slos/).
- **Implementing Service Level Objectives** — Alex Hidalgo.
- **Observability Engineering**, 2nd ed. — Honeycomb (2026, 32 new chapters covering cost, governance, AI).

Blogs to Follow

- **Honeycomb** — honeycomb.io/blog (Charity Majors et al.)
- **Grafana Labs** — grafana.com/blog (Mimir, Loki, Tempo, Pyroscope deep dives)
- **Isovalent / Cilium** — isovalent.com/blog (eBPF networking)
- **Polar Signals** — polarsignals.com/blog (continuous profiling)
- **Last9** — last9.io/blog (observability architecture comparisons)

- **VictoriaMetrics** — victoriametrics.com/blog
- **sanj.dev** — Prometheus scaling deep dives
- **sre.google** — Google SRE workbook chapters

Talks (YouTube)

- **KubeCon Observability Track** — annual playlist (CNCF).
- **SRECon Talks** (USENIX).
- **Charity Majors — Observability for Engineers**.
- **Liz Fong-Jones — Adopting SLOs**.
- **Frederic Branczyk — eBPF profiling**.
- **Bartlomiej Plotka — Thanos architecture**.
- **Tom Wilkie — Mimir scaling**.

Hands-on Labs

- **Grafana LGTM stack** docker-compose — local end-to-end.
- **OpenLLMetry quickstart** — instrument an OpenAI app in 10 min.
- **Pyroscope flame graph demo** — profile a sample app.
- **Chaos Mesh on KinD** — run a pod-kill chaos experiment.
- **Sloth** (github.com/slok/sloth) — generate SLO recording & alerting rules from YAML.
- **Pyrra** (github.com/pyrra-dev/pyrra) — K8s-native SLO operator.
- **Cilium Hubble lab** — visualize flows on a small K8s cluster.
- **VictoriaMetrics single-binary** — try as Prom long-term storage.

Practice Platforms

- **Killercoda observability scenarios** — free interactive labs.
- **KodeKloud — Prometheus Certified Associate**.
- **Linux Foundation LFS148** — Intro to OpenTelemetry (free).
- **O'Reilly Learning** — Observability Engineering video course.

> **Final note:** these advanced topics are how you signal staff/principal-level depth. Don't memorize — practice *explaining* the trade-offs out loud. The interviewer wants to hear how you think under pressure, not what you've read.